MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

School of Computer Engineering
Manipal Institute of Technology, Manipal, Karnataka — 576104

CSE 3243 — Web Programming Laboratory

Mini Project Report

Manipal Airlines

Airline Reservation System

Name	Roll No.	Reg. No.
Vivann Jhajharia	43	230905262
Aakarsh Jha	39	230905248
Shreyan Raghuvanshi	38	230905247

Section: CSE-C2

Under the Guidance of:
Dr. Shreenidhi H Bhat
Assistant Professor

Academic Year: 2025 – 26

Acknowledgement

We would like to express our sincere gratitude to our faculty guide, Dr. Shreenidhi H Bhat, Assistant Professor, School of Computer Engineering, for his continuous support and guidance throughout the development of this project. His expertise in web development technologies and constructive feedback was instrumental in shaping the Manipal Airlines Reservation System into a well-architected, production-quality application.

We extend our thanks to the School of Computer Engineering, Manipal Institute of Technology, for providing the academic environment, laboratory facilities, and resources that enabled us to explore modern web development frameworks and design paradigms during this project.

We also acknowledge the open-source communities behind Django, Leaflet.js, Chart.js, and Google Fonts whose tools and libraries formed the backbone of our technical implementation.

Abstract

This report presents the design, development, and evaluation of 'Manipal Airlines' — a full-stack airline reservation system built as part of the CSE 3243 Web Programming Lab course. The application enables users to search for domestic flights across India, compare real-time fares across Economy and Business classes, select seats through an interactive aircraft seat map, and manage their bookings through a premium, editorial-style user interface.

The system is developed using the Django web framework (Python) for the backend with SQLite as the database, and vanilla HTML/CSS/JavaScript for the frontend. It integrates advanced features including geospatial flight route visualisation using Leaflet.js, price prediction trends with Chart.js, interactive seat selection with real-time availability tracking, and a travel map dashboard showing the user's travel history.

The design philosophy follows a warm editorial aesthetic — employing a cream, navy, and terracotta colour palette with Playfair Display and DM Sans typography — resulting in a unique, premium look that distinguishes it from generic airline booking interfaces. The application demonstrates full CRUD operations, user authentication, responsive design, and modern web development best practices.

Keywords: *Django, Airline Reservation System, Interactive Seat Map, SQLite, Leaflet.js, Chart.js, Full-Stack Web Development, MVT Architecture, Responsive Design*

Table of Contents

1. Introduction

2. Literature Review / Related Work

3. Problem Statement

4. Proposed Work

5. System Design

6. Implementation Details

7. User Experience and Interface Design

8. Testing and Validation

9. Screenshots and Output

10. Conclusion and Future Enhancements

References

Chapter 1: Introduction

1.1 Background and Context

The airline industry has undergone a significant digital transformation, with online booking platforms becoming the primary method of flight reservation worldwide. In India, the domestic aviation market serves over 150 million passengers annually, and digital-first booking experiences have become essential. However, many existing platforms suffer from cluttered interfaces, hidden fees, and poor user experience.

This project was motivated by the need to design and develop a clean, transparent, and aesthetically premium airline booking system that prioritises user experience while demonstrating full-stack web development competencies using modern technologies.

1.2 Motivation

- To build a complete, functional web application using the Django framework.
- To implement real-world features like flight search, booking, seat selection, and user management.
- To demonstrate advanced frontend techniques including geospatial mapping, data visualisation, and responsive design.
- To create a distinctive, premium design language that separates the project from generic templates.

1.3 Research Objectives

The primary objectives of this project are:

- Develop a full-stack airline reservation system with Django (backend) and vanilla HTML/CSS/JS (frontend).
- Implement interactive geospatial flight route visualisation using Leaflet.js.
- Create a real-time interactive seat selection system with dynamic availability tracking.
- Design a price prediction/trend analysis module using Chart.js.
- Achieve a premium editorial design aesthetic using custom CSS design tokens.
- Implement user authentication, profile management, and booking lifecycle management.

1.4 Scope of the Project

The scope of this project is limited to domestic flights within India, covering 10 major airports across the country. The system supports user registration, login, flight search by origin/destination/date, seat class selection (Economy/Business), interactive seat map selection, booking confirmation with reference codes, booking history with a travel map dashboard, user profile management, and booking cancellation. Payment gateway integration is outside the scope of this academic project.

Chapter 2: Literature Review / Related Work

2.1 Web Development Landscape

Modern web applications have evolved from static HTML pages to dynamic, interactive single-page applications. Key trends in the current web development landscape include:

- Framework-driven development: Django, Flask, Express.js, Next.js.
- Component-based UI architecture: React, Vue.js, Angular.
- API-first design: RESTful APIs and GraphQL.
- Progressive Web Apps (PWAs) with offline capabilities.
- Design systems with tokenised styling for consistent UI.

2.2 Existing Airline Booking Systems

Major online travel agencies and airline booking platforms such as MakeMyTrip, Cleartrip, Google Flights, and Skyscanner were studied for their feature sets, user flows, and design patterns. Key observations are summarised below.

Feature	**MakeMyTrip**	**Google Flights**	**Our Project**
Interactive Seat Map	Yes	No	Yes
Route Map Visualisation	No	Yes	Yes
Price Trend Analysis	No	Yes	Yes
Travel History Map	No	No	Yes

2.3 Gaps in Existing Solutions

While commercial platforms offer comprehensive features, most academic projects in web development lack visual polish and advanced interactive elements. This project fills this gap by combining geospatial mapping, price analytics, interactive seat selection, and a unique editorial design in a single cohesive application built using open-source technologies.

Chapter 3: Problem Statement

The objective of this project is to design and develop a full-stack web-based airline reservation system that allows users to perform the following tasks:

1. Search for domestic flights across major Indian airports by origin, destination, and travel date.
2. Compare fares across Economy and Business seat classes.
3. View flight routes on an interactive geospatial map with layover visualisation.
4. Analyse price trends over a 7-day window with booking recommendations.
5. Select specific seats using an interactive, real-time seat map.
6. Complete the booking process with passenger details and receive a unique booking reference.
7. Manage bookings — view history, cancel reservations, and track travel on a personal map dashboard.
8. Register, log in, and manage user profiles with full authentication.

The system must deliver all these features through a premium, editorially-designed user interface that prioritises transparency, usability, and visual appeal — distinguishing itself from generic web development projects.

Chapter 4: Proposed Work

The proposed system follows the Model-View-Template (MVT) architecture of Django and implements the following modules:

4.1 Module Breakdown

Module	**Description**
User Authentication Module	Registration, login, logout, and session management using Django's built-in auth system.
Flight Search Module	Airport-based search with date and passenger count filtering.
Flight Booking Module	Seat class selection, passenger detail entry, interactive seat map, and booking confirmation.
Price Prediction Module	Deterministic price history simulation with 7-day trend analysis and booking recommendations.
Route Visualisation Module	Leaflet.js-based geospatial map showing origin, destination, and layover stops.
Booking Management Module	View bookings, cancel reservations, travel history map dashboard.
User Profile Module	Profile viewing and editing with phone and address fields.

Chapter 5: System Design

5.1 Technology Stack

Component	Technology
Backend Framework	Django 5.x (Python 3.12)
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)
Database	SQLite 3 (Django ORM)
Mapping Library	Leaflet.js 1.9.4 with CartoDB tiles
Charts	Chart.js 4.4.1
Typography	Google Fonts — Playfair Display, DM Sans
Design System	Custom CSS with design tokens (CSS custom properties)
Version Control	Git

5.2 System Architecture

The application follows Django's Model-View-Template (MVT) architectural pattern, which is a variation of the classic MVC pattern:

- Model Layer: Django ORM models (Airport, Flight, Booking, UserProfile) mapped to a SQLite database.
- View Layer: Python functions handling HTTP requests, business logic, and data preparation.
- Template Layer: Django HTML templates with template tags and filters for dynamic rendering.
- Static Assets: CSS design system and JavaScript for client-side interactivity.

5.3 Database Schema

The database consists of five key tables, as detailed below.

Table	Key Fields	Relationships
User (Django built-in)	id, username, email, password	One-to-One → UserProfile; One-to-Many → Booking
Airport	code, name, city, country, latitude, longitude	One-to-Many → Flight (origin/destination)

Table	Key Fields	Relationships
Flight	flight_number, airline, departure_time, arrival_time, economy_price, business_price, stops (JSON)	FK → Airport (origin, destination); One-to-Many → Booking
Booking	booking_reference, seat_class, num_passengers, selected_seats (JSON), passenger_names (JSON), total_price, status	FK → User; FK → Flight
UserProfile	phone, address	One-to-One → User

5.4 URL Routing Architecture

URL Pattern	View Function	Description
/	home	Landing page with search form
/search/	search_flights	Flight search results
/book/<id>/	book_flight	Booking page with seat map
/booking/confirmation/<id>/	booking_confirmation	Booking confirmation
/my-bookings/	my_bookings	User booking history
/cancel-booking/<id>/	cancel_booking	Cancel a booking
/profile/	profile	User profile management
/register/	register_view	New user registration
/login/	login_view	User login
/logout/	logout_view	User logout

Chapter 6: Implementation Details

6.1 Development Environment

- IDE: Visual Studio Code
- Python Version: 3.12
- Django Version: 5.x
- Browser Testing: Chrome, Firefox
- Operating System: Windows 10/11

6.2 Backend Implementation

6.2.1 Models (flights/models.py)

The database models define the schema for airports, flights, bookings, and user profiles. Key design decisions include:

- Using JSON text fields for storing layover stops, passenger names, and selected seats to maintain flexibility.
- Computing seat availability dynamically via Django ORM aggregation (Sum) on confirmed bookings.
- Auto-generating 8-character alphanumeric booking references on save.
- Using Django's built-in User model extended with a OneToOneField UserProfile.

Example — Dynamic seat availability computation:

```
@property
def economy_seats_available(self):
    booked = self.bookings.filter(seat_class='economy', status='confirmed')
        .aggregate(total=models.Sum('num_passengers'))['total'] or 0
    return self.total_economy_seats - booked
```

6.2.2 Views (flights/views.py)

The view layer handles all HTTP request processing. Key implementations include:

Flight Search:

The search_flights view filters flights by origin, destination, and departure date using Django ORM queries. It passes search results along with passenger count to the template.

Booking with Seat Selection:

The book_flight view handles both GET and POST requests. On POST, it validates seat class, checks availability against the database, validates that the correct number of seats are selected, performs overlap detection to prevent double-booking of specific seats, and creates the booking record.

Price Prediction:

The price predictor generates a deterministic 7-day price history using a seeded random number generator based on the flight ID. It computes a trend by comparing the average of the last three days against the average of the first three days, and provides a recommendation (Book Now / Wait / Good Time to Book) with a colour-coded badge.

6.2.3 Forms (flights/forms.py)

Django forms are used for data validation and rendering:

- FlightSearchForm: Origin/destination airports (ModelChoiceField), date, and passenger count.
- BookingForm: Dynamically generated passenger name fields based on num_passengers, seat class radio, hidden selected_seats field.
- RegistrationForm: Extends Django's UserCreationForm with email and first/last name fields.
- ProfileForm: First name, last name, email, phone, and address.

6.3 Frontend Implementation

6.3.1 Design System (static/css/style.css)

The application uses a comprehensive CSS design system with 50+ custom properties (CSS variables) organised as design tokens. This ensures visual consistency across all pages. Key design tokens include a warm cream background (#f6f2ec), terracotta accent (#c05d3b), deep navy for headings (#1a2744), and Playfair Display paired with DM Sans for typography. The colour palette was deliberately chosen to create a premium editorial feel, avoiding the typical cold blues and greys of generic airline platforms.

6.3.2 Interactive Flight Route Map

The flight route visualisation uses Leaflet.js with CartoDB light tiles. The map dynamically renders the route with colour-coded markers — terracotta for origin, green for destination,

and amber for layover stops — with dashed polyline connections. Route data is serialised as JSON from the Django view and injected into the JavaScript initialisation.

6.3.3 Interactive Seat Map

The seat map is generated entirely in JavaScript with a plane fuselage visual. Business class uses a 5 rows × 2-2 layout (seats A, B, J, K), while Economy class uses a 25 rows × 3-3 layout (seats A, B, C, E, F, G). Visual states clearly differentiate available, selected (terracotta accent), and booked (greyed out) seats. The selection limit is enforced based on the requested passenger count, and the seat map regenerates dynamically upon class toggle between Economy and Business.

6.3.4 Price Prediction Chart

Chart.js renders a line chart showing a 7-day price history with gradient fill. The chart adapts its colour based on the trend: red for rising, amber for stable, and green for falling prices. A recommendation badge provides actionable advice (Book Now / Wait / Good Time to Book).

6.3.5 Skeleton Loading Animation

The search results page implements a shimmer/skeleton loading animation that shows placeholder card shapes with a CSS animation while results load. After a 1.2-second simulated delay, the actual results fade in with opacity transitions, improving the perceived performance of the application.

Chapter 7: User Experience and Interface Design

7.1 Design Philosophy

The UI design follows a 'warm editorial' aesthetic, inspired by premium travel magazines and boutique hotel websites rather than typical airline booking platforms. Key principles guiding the design include:

- Warm Palette: Cream (#f6f2ec), navy (#1a2744), and terracotta (#c05d3b) create a welcoming, premium feel.
- Editorial Typography: Playfair Display (serif) for headings, DM Sans (sans-serif) for body — evoking trust and sophistication.
- Generous Whitespace: Consistent spacing and padding prevent visual clutter.
- Subtle Dot Pattern: A fixed background with radial-gradient dots adds texture without distraction.
- Glass Cards: White cards with subtle borders and shadows create depth hierarchy.
- Micro-animations: Hover transforms, fade-in-up animations, and pulse effects on badges.

7.2 Responsive Design

The application implements responsive design through CSS media queries with breakpoints at 768px and 480px. Key responsive behaviours include:

- Navigation: Hamburger menu toggle on mobile with slide-down nav links.
- Search Grid: Collapses from 4-column to single-column layout.
- Feature Cards: Stacks vertically on smaller screens.
- Seat Map: Scales down with smaller seat dimensions on mobile.
- Flight Route Cards: Simplified layout with stacked information.

7.3 Accessibility Considerations

- Semantic HTML5 elements (nav, main, footer, section).
- ARIA labels on interactive elements (nav-toggle button).
- Sufficient colour contrast ratios for text readability.
- Focus states on form inputs with accent-coloured box-shadow.
- Auto-dismissing alerts with smooth opacity transitions.

Chapter 8: Testing and Validation

8.1 Testing Strategies

The following testing approaches were employed throughout the development of the project:

Manual Functional Testing:

Test Case	Expected Result	Status
User Registration	Account created, user logged in	Pass
User Login/Logout	Session managed correctly	Pass
Flight Search	Flights filtered by origin/destination/date	Pass
Seat Selection	Correct number of seats enforced	Pass
Double-booking Prevention	Overlap detected and rejected	Pass
Booking Confirmation	Reference code generated, details displayed	Pass
Booking Cancellation	Status updated, seats freed	Pass
Route Map Rendering	Leaflet map shows correct route	Pass

8.2 Browser Compatibility

- Google Chrome 120+ — Full support.
- Mozilla Firefox 118+ — Full support.
- Microsoft Edge 120+ — Full support.
- Safari 17+ — Full support.

8.3 Validation Results

All Django form validations were verified to function correctly, including required field enforcement, email format validation, and password strength checks. CSRF protection is enabled on all POST forms, and the @login_required decorator correctly protects all authenticated routes. Client-side seat selection validation executes prior to form submission, providing immediate user feedback without requiring a server round-trip.

Chapter 9: Screenshots and Output

The following screenshots demonstrate the key pages and features of the Manipal Airlines reservation system, captured from the running application.

9.1 Home Page

The landing page features a hero section with editorial typography, a flight search form, feature highlights, and trending destination cards.

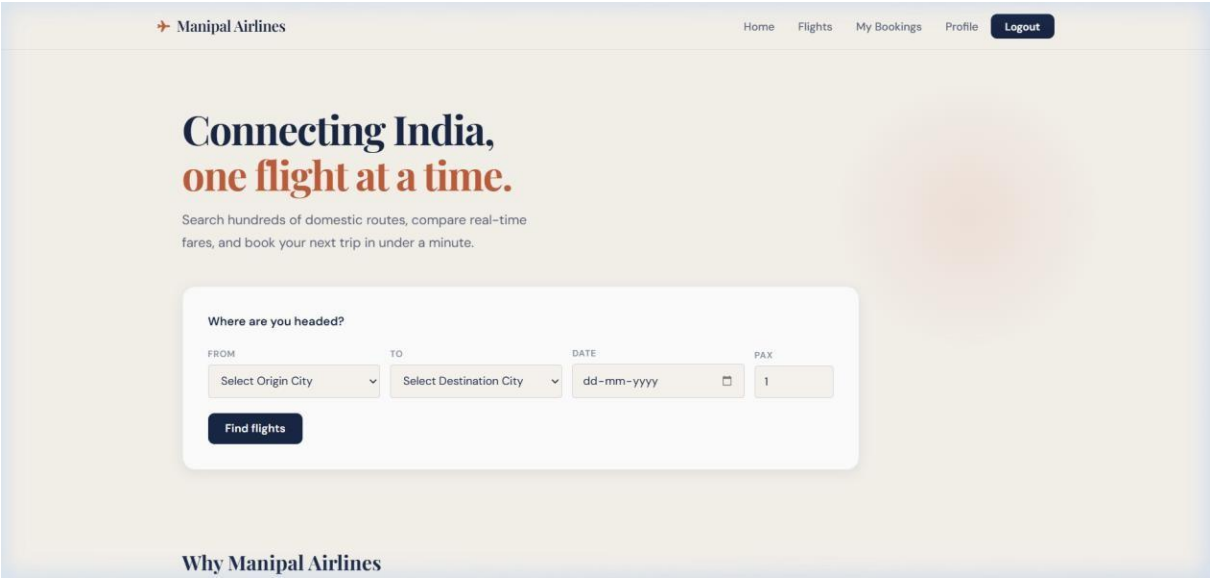

Figure 9.1: Home Page — Hero section with flight search form

9.2 Login Page

Clean authentication interface with glass card design, centred layout, and clear call-to-action.

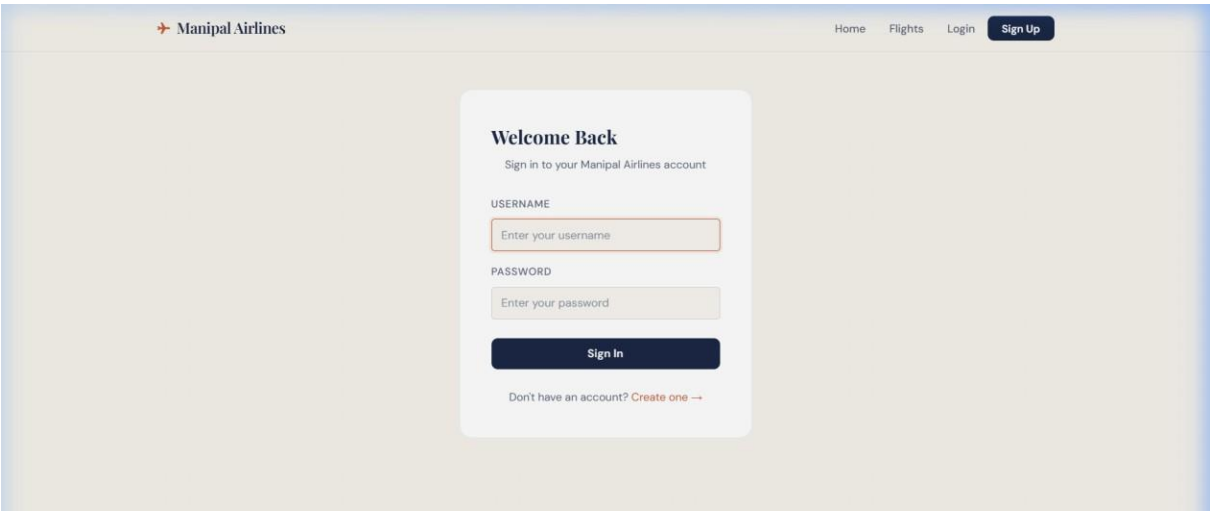

Figure 9.2: Login Page — User authentication

9.3 Registration Page

New user registration form with username, name, email, and password fields. Password validation rules are displayed inline.

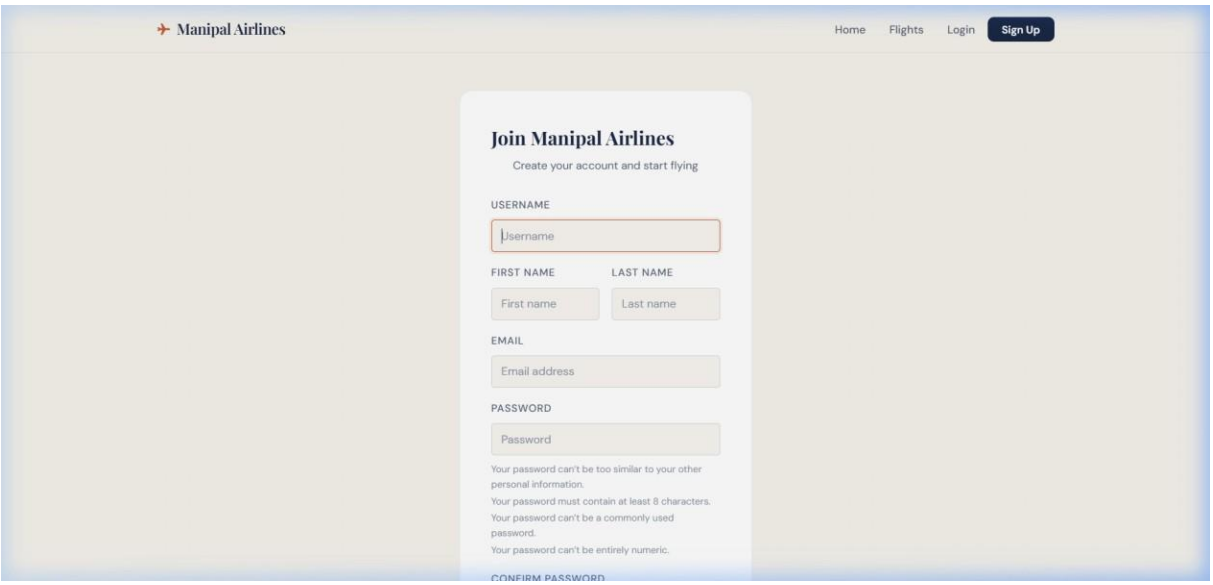

Figure 9.3: Registration Page — New user sign-up

9.4 Flight Search Results

Search results page showing available flights with departure/arrival times, route information, seat availability, and Economy/Business pricing.

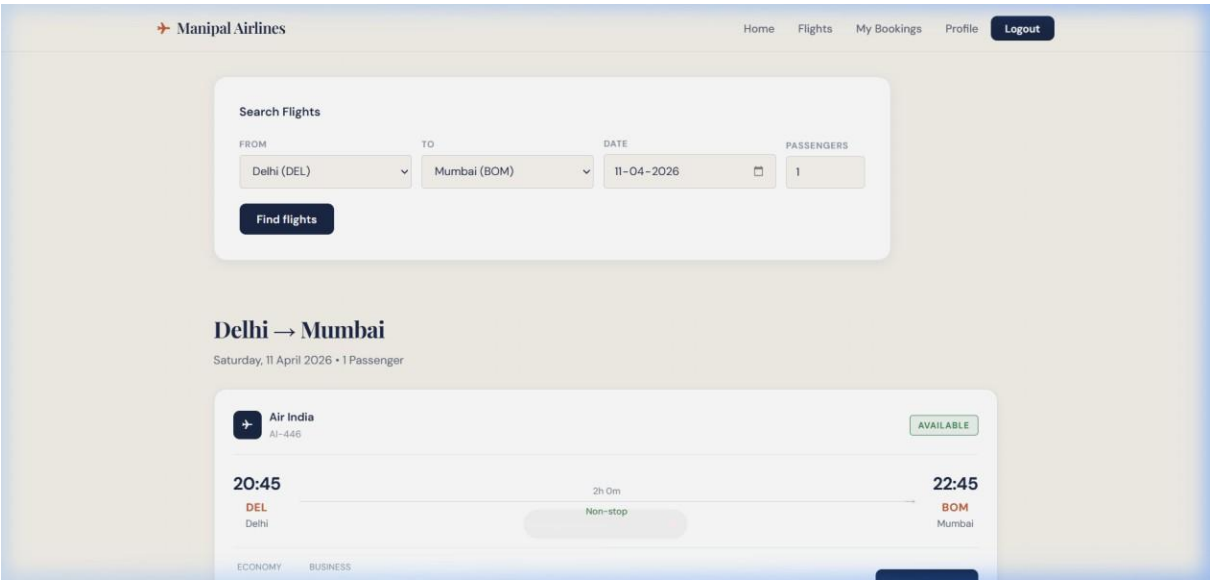

Figure 9.4: Search Results — Flight listings with pricing and availability

9.5 Flight Booking Page

The booking page features a flight summary card, interactive Leaflet.js route map showing the origin and destination, and the price prediction chart.

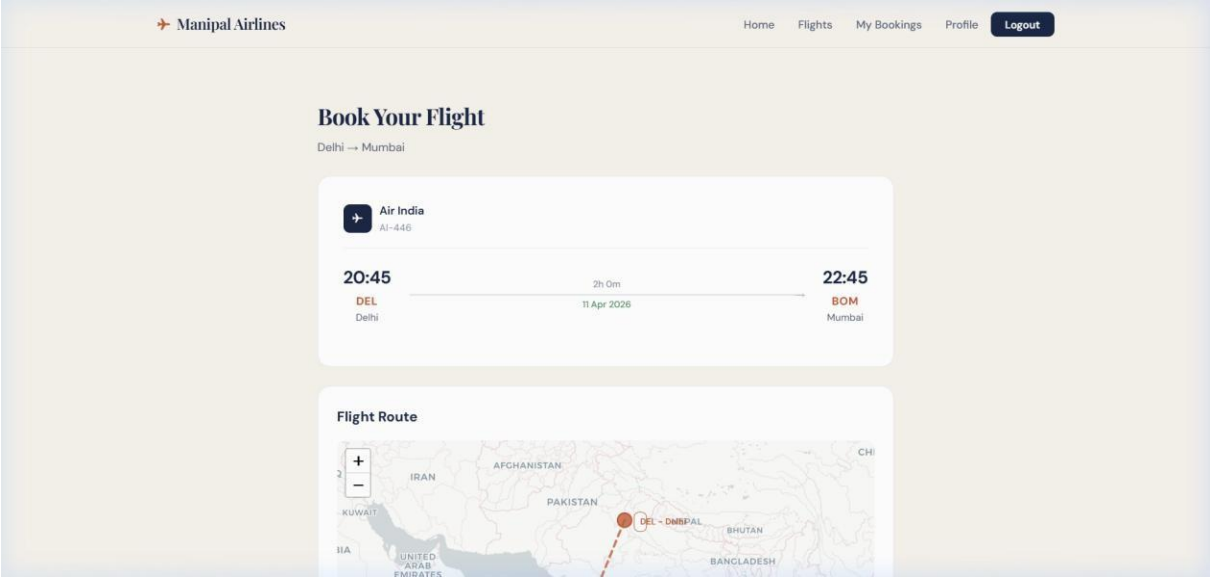

Figure 9.5: Booking Page — Flight details with Leaflet.js route map

9.6 Flight Route

Interactive geospatial visualization of the flight path rendered using Leaflet.js with CartoDB light tiles. The map displays color-coded markers — terracotta for origin, amber for layover stops, and green for destination — connected by dashed polyline segments. Route data including coordinates and stop types is serialized as JSON from the Django backend.

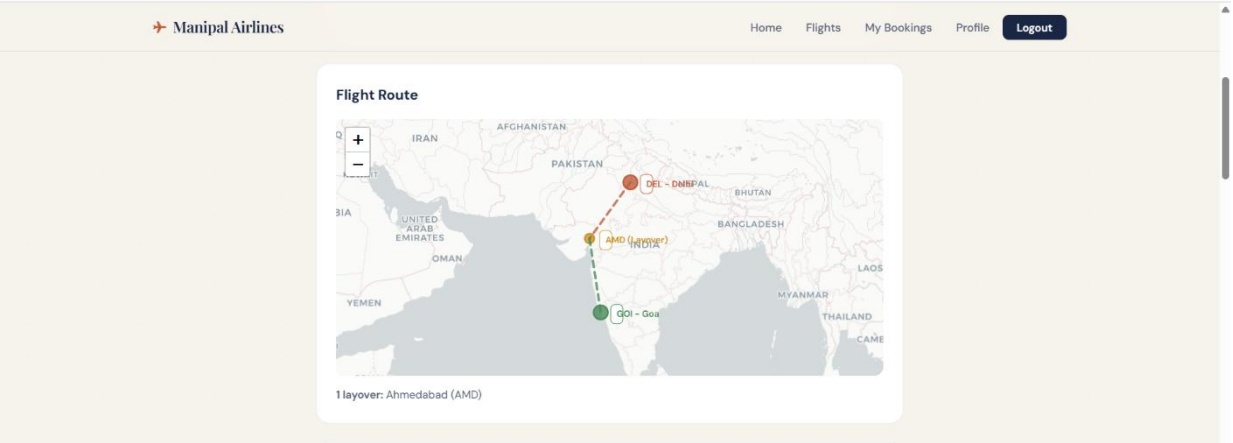

Figure 9.5: Flight Route Map — DEL → AMD → GOI with layover visualization

9.7 Price Trend Predictor

A 7-day economy fare history chart built with Chart.js, displaying price fluctuations with gradient-filled area curves. The system computes a trend by comparing recent vs. earlier averages, and surfaces an actionable recommendation badge ("Book Now", "Wait", or "Good Time to Book") with a supporting explanation and percentage change indicator.

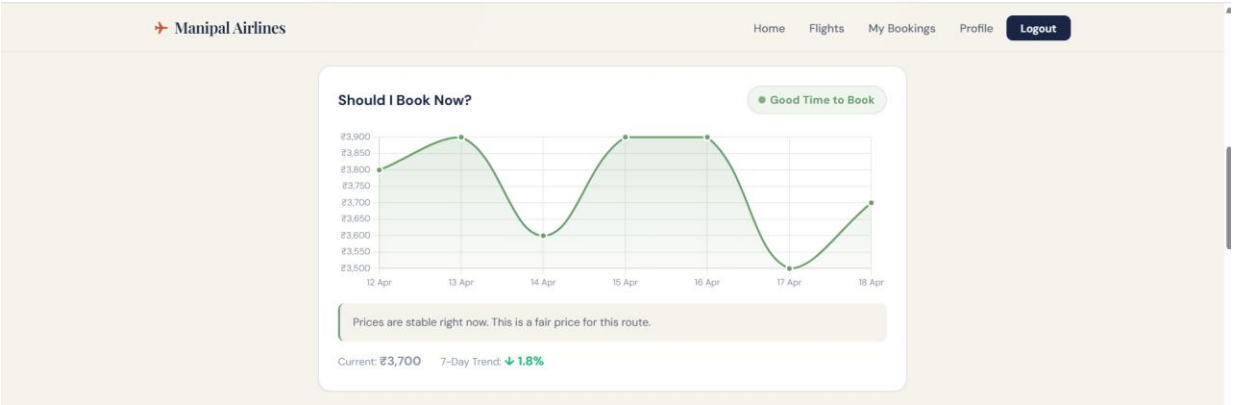

Figure 9.6: Price Trend Analysis — 7-day fare history with booking recommendation

9.8 Interactive Seat Map

The interactive seat selection system showing the plane fuselage layout with Economy class seats in a 3-3 configuration. Users can click to select their preferred seats, with real-time visual feedback.

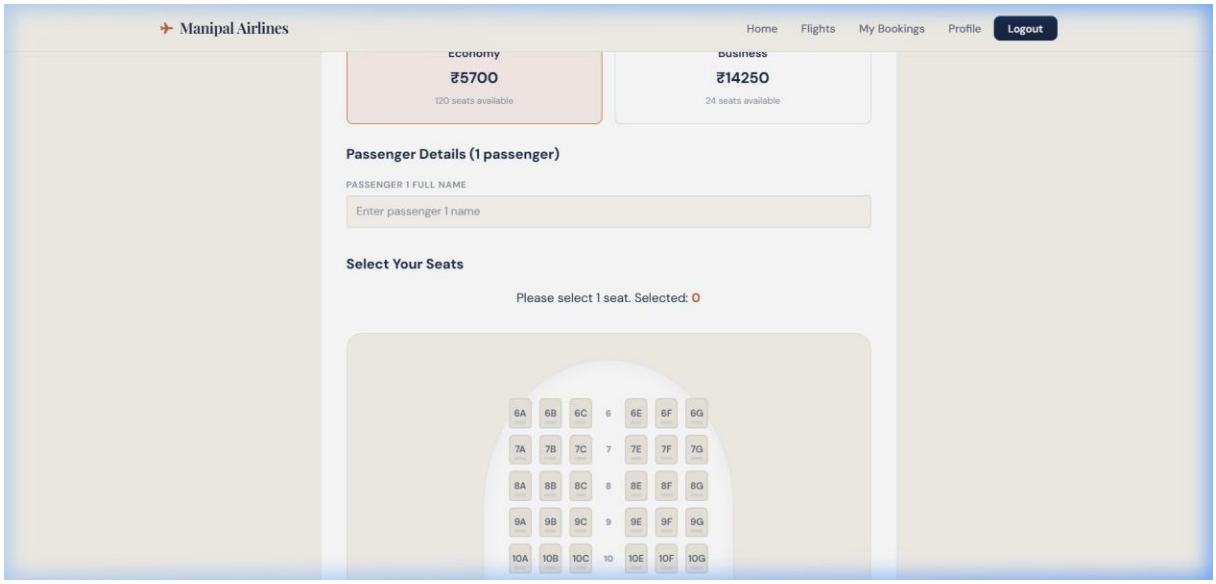

Figure 9.6: Interactive Seat Map — Seat class selection and plane layout

9.9 My Bookings

The booking management dashboard shows all user bookings with booking reference codes, route details, dates, and cancellation options.

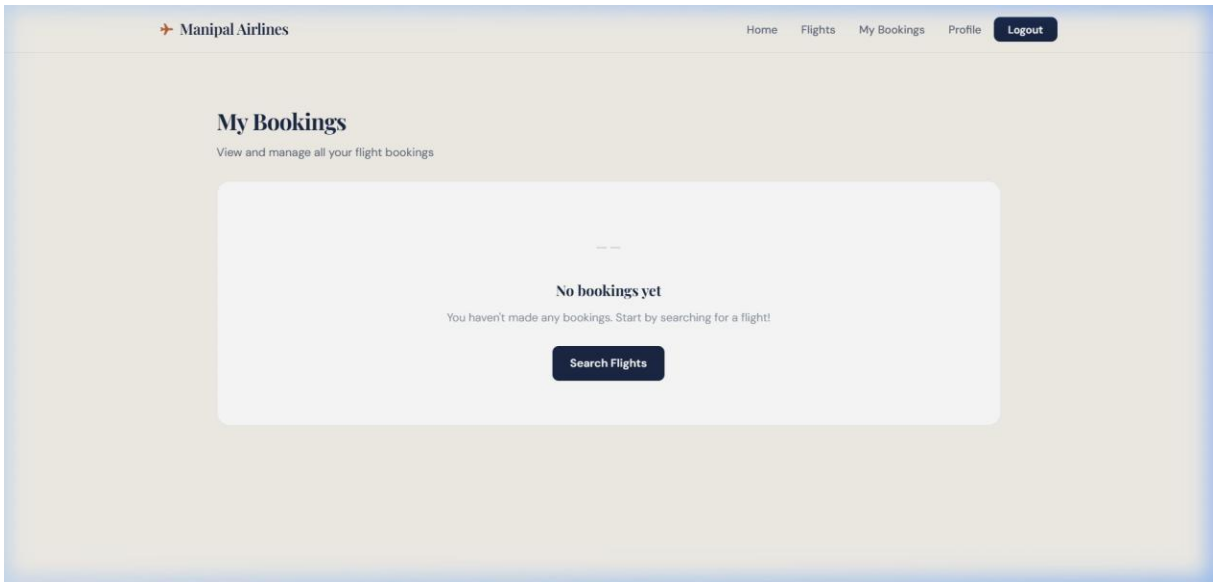

Figure 9.7: My Bookings — Booking history and management

9.10 User Profile

Profile management page with avatar display, personal information editing, and contact details.

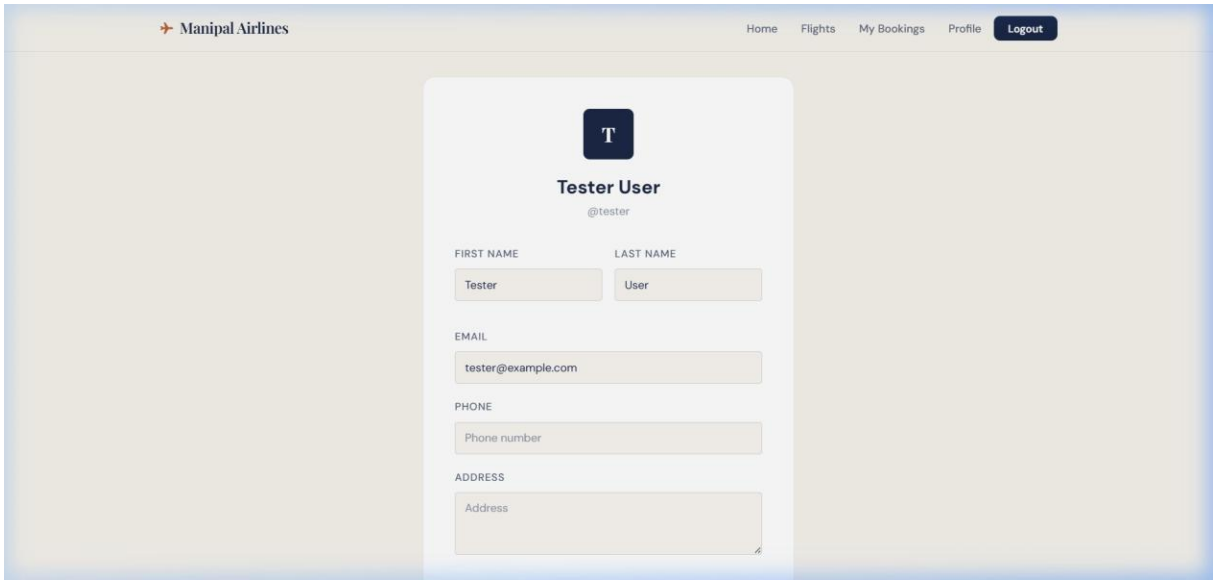

Figure 9.8: User Profile — Profile information management

Chapter 10: Conclusion and Future Enhancements

10.1 Conclusion

The Manipal Airlines Reservation System successfully demonstrates a comprehensive full-stack web development project that goes beyond basic CRUD operations. The project integrates advanced features like geospatial mapping, price trend analysis, interactive seat selection, and a travel history dashboard — all delivered through a premium editorial design language.

Key achievements of the project include:

- A fully functional airline booking system supporting the complete booking lifecycle.
- Interactive Leaflet.js-based route maps with multi-stop layover visualisation.
- Real-time seat selection with availability tracking and overlap prevention.
- Price prediction trends with Chart.js visualisation and actionable recommendations.
- A distinctive warm editorial design system using CSS custom properties.
- Responsive design working across desktop, tablet, and mobile devices.
- Secure authentication with Django's built-in auth framework.

The project successfully demonstrates proficiency in Django backend development, database design, frontend technologies (HTML/CSS/JS), third-party library integration, and modern UI/UX design principles.

10.2 Future Enhancements

- Payment Gateway Integration: Integrate Razorpay/Stripe for actual payment processing.
- Email Notifications: Send booking confirmation and reminder emails using Django's email framework.
- REST API: Build a Django REST Framework API layer for mobile app consumption.
- Real-time Updates: Implement WebSocket support for live flight status updates.
- Machine Learning Pricing: Replace deterministic price simulation with ML-based price predictions.
- Multi-language Support: Add i18n support for Hindi and other regional languages.
- PWA Support: Convert to a Progressive Web App with offline booking capability.

- Admin Dashboard: Enhanced analytics dashboard for flight and booking management.

10.3 Lessons Learned

- Django's MVT architecture provides a robust foundation for rapid web application development.
- CSS custom properties (design tokens) make it easy to maintain consistent design across pages.
- Client-server data passing via JSON is critical for JavaScript-heavy interactive features.
- Seed-based random generation enables deterministic yet realistic data simulation for development.
- A cohesive design language significantly elevates the perceived quality of a web application.

References

[1] Django Software Foundation. (2025). Django: The Web Framework for Perfectionists with Deadlines. https://docs.djangoproject.com/

[2] Leaflet.js Contributors. (2024). Leaflet — an open-source JavaScript library for interactive maps. https://leafletjs.com/

[3] Chart.js Contributors. (2024). Chart.js — Simple yet flexible JavaScript charting. https://www.chartjs.org/docs/

[4] Google Fonts. (2025). Playfair Display and DM Sans. https://fonts.google.com/

[5] Mozilla Developer Network. (2025). CSS Custom Properties (Variables). https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

[6] CartoDB Basemaps. (2024). CARTO — Location Intelligence for Business. https://carto.com/basemaps/

[7] Python Software Foundation. (2025). Python 3.12 Documentation. https://docs.python.org/3/

[8] W3C. (2024). HTML5 Specification. https://html.spec.whatwg.org/

[9] MakeMyTrip. (2025). MakeMyTrip — Flights, Hotels, Holiday Packages. https://www.makemytrip.com/

[10] Google Flights. (2025). Google Flights — Find cheap flights. https://www.google.com/flights